

TUIA

BASES DE DATOS I – 1 SEM2026 – R1

- Docentes:

- Fernando Roldan
- Cristián Gallo
- Lara Della Ceca

- FCEIA-UNR

TUIA - BASES DE DATOS I

PROCEDIMIENTOS ALMACENADOS - STORED PROCEDURES

Un procedimiento almacenado de SQL Server es un grupo de una o varias instrucciones Transact-SQL.

Los procedimientos se asemejan a las construcciones de otros lenguajes de programación, porque pueden:

- Aceptar parámetros de entrada y devolver varios valores en forma de parámetros de salida al programa que realiza la llamada.
- Contener instrucciones de programación que realicen operaciones en la base de datos. Entre otras, pueden contener llamadas a otros procedimientos.
- Devolver un valor de estado a un programa que realiza una llamada para indicar si la operación se ha realizado correctamente o se han producido errores, y el motivo de estos.

TUIA - BASES DE DATOS I

PROCEDIMIENTOS ALMACENADOS - STORED PROCEDURES

Ventajas de usar procedimientos almacenados

En la siguiente lista se describen algunas de las ventajas que brinda el uso de procedimientos.

- Tráfico de red reducido entre el cliente y el servidor
- Mayor seguridad
- Reutilización del código
- Mantenimiento más sencillo
- Rendimiento mejorado (Se compila el plan de ejecución solo una vez)

Mas info: [Procedimientos almacenados \(motor de base de datos\) - SQL Server | Microsoft Learn](#)

TUIA - BASES DE DATOS I

PROCEDIMIENTOS ALMACENADOS - STORED PROCEDURES

Crear un SP

[Crear un procedimiento almacenado - SQL Server | Microsoft Learn](#)

Se pueden crear mediante SQL Server Managment Studio o mediante T-SQL

TUIA - BASES DE DATOS I

PROCEDIMIENTOS ALMACENADOS - STORED PROCEDURES

Crear un SP - CREATE PROCEDURE

```
USE [Northwind]
```

```
GO
```

```
CREATE PROCEDURE usp_GetEmployeesTest
```

```
    @LastName nvarchar(50),
```

```
    @FirstName nvarchar(50)
```

```
AS
```

```
    SET NOCOUNT ON;
```

```
    SELECT Title, FirstName, LastName
```

```
    FROM Employees
```

```
    WHERE FirstName = @FirstName AND LastName = @LastName;
```

```
GO
```

TUIA - BASES DE DATOS I

PROCEDIMIENTOS ALMACENADOS - STORED PROCEDURES

Llamada a un SP - EXECUTE

Se utiliza la palabra clave EXECUTE o EXEC y deben especificarse los parametros

```
USE [Northwind]
```

```
GO
```

```
EXECUTE usp_GetEmployeesTest 'Davolio', 'Nancy'
```

```
GO
```

```
EXECUTE usp_GetEmployeesTest 'Nancy', 'Davolio'
```

```
GO
```

```
EXECUTE usp_GetEmployeesTest @FirstName='Nancy', @LastName='Davolio'
```

```
GO
```

TUIA - BASES DE DATOS I

PROCEDIMIENTOS ALMACENADOS - STORED PROCEDURES

Modificar un SP – ALTER PROCEDURE

```
USE [Northwind]
GO

SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

ALTER PROCEDURE [dbo].[usp_GetEmployeesTest]
    @LastName nvarchar(50),
    @FirstName nvarchar(50)
AS
    SET NOCOUNT ON;
    SELECT FirstName, LastName, Title, HomePhone
    FROM Employees
    WHERE FirstName = @FirstName AND LastName = @LastName;
```

TUIA - BASES DE DATOS I

PROCEDIMIENTOS ALMACENADOS - STORED PROCEDURES

Eliminar un SP – DROP PROCEDURE

```
DROP PROCEDURE [<stored procedure name>];
```

```
GO
```

```
USE [Northwind]
```

```
GO
```

```
DROP PROCEDURE [dbo].[usp_GetEmployeesTest]
```


TUIA - BASES DE DATOS I

PROCEDIMIENTOS ALMACENADOS - STORED PROCEDURES

Parámetros

Los parámetros se usan para intercambiar datos entre las funciones y los procedimientos almacenados y la aplicación o la herramienta que llamó a la función o al procedimiento almacenados:

- Los parámetros de entrada permiten a quien realiza la llamada pasar un valor de datos a la función o al procedimiento almacenado.
- Los parámetros de salida permiten al procedimiento almacenado devolver un valor de datos o variable de cursor a quien realizó la llamada. Las funciones definidas por el usuario no pueden especificar parámetros de salida.
- Cada procedimiento almacenado devuelve un código de retorno de tipo entero a quien realiza la llamada. Si el procedimiento almacenado no establece explícitamente un valor para el código de retorno, éste es 0.

TUIA - BASES DE DATOS I

PROCEDIMIENTOS ALMACENADOS - STORED PROCEDURES

Parámetros

```
CREATE PROCEDURE usp_SampleProcedure
@EmployeeIDParm INT,
    @Total INT OUTPUT
AS
DECLARE @ErrorSave INT
SET @ErrorSave = 0

-- SELECT usando el parametro de entrada
SELECT FirstName, LastName, Title
FROM Employees
WHERE EmployeeID = @EmployeeIDParm
```

TUIA - BASES DE DATOS I

PROCEDIMIENTOS ALMACENADOS - STORED PROCEDURES

Parámetros

```
-- Guardar @@ERROR distinto de 0.  
IF (@@ERROR <> 0)  
    SET @ErrorSave = @@ERROR  
  
-- Setear un valor en el parametro de salida.  
SELECT @Total = COUNT(*)  
FROM Orders  
WHERE EmployeeID = @EmployeeIDParm;  
  
SELECT @Total  
IF (@@ERROR <> 0)  
    SET @ErrorSave = @@ERROR  
  
-- Devuelve 0 si nignun SELECT dio error; de otra forma devuelve el ultimo error.  
RETURN @ErrorSave  
GO
```

TUIA - BASES DE DATOS I

PROCEDIMIENTOS ALMACENADOS - STORED PROCEDURES

Parámetros

```
DECLARE @ReturnCode INT
```

```
DECLARE @Total1 INT
```

```
EXEC @ReturnCode = usp_SampleProcedure 5, @Total1 OUTPUT
```

```
SELECT @ReturnCode
```

```
SELECT @Total1
```

TUIA - BASES DE DATOS I

VISTAS

- Una vista es una tabla virtual cuyo contenido está definido por una consulta.
- Es un conjunto de columnas y filas con nombre, pero no existe como conjunto almacenado en la BD
- Las filas y las columnas de datos proceden de tablas a las que se hace referencia en la consulta que define la vista y se producen de forma dinámica cuando se hace referencia a la vista.

TUIA - BASES DE DATOS I

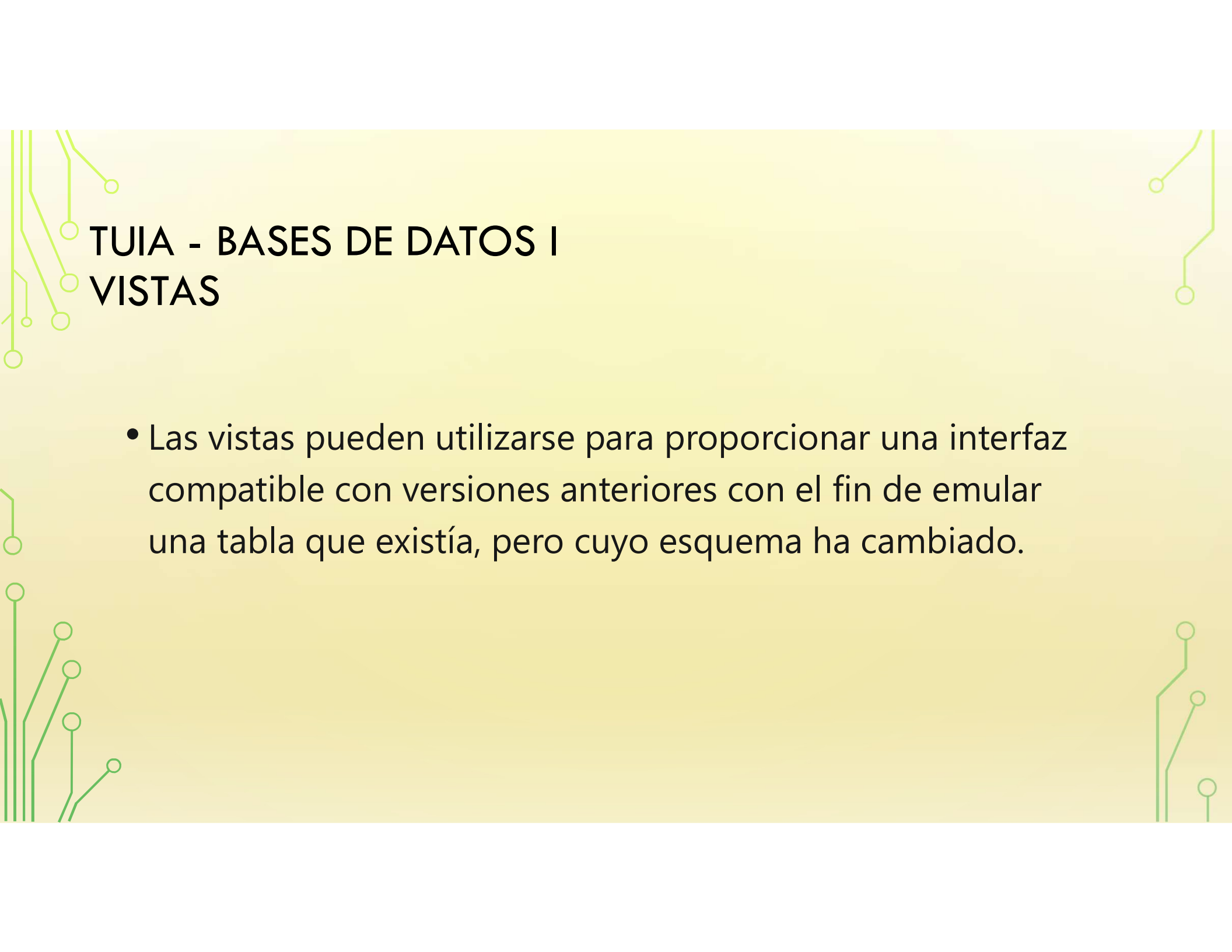
VISTAS

- Una vista actúa como filtro de las tablas subyacentes a las que se hace referencia en ella.
- La consulta que define la vista puede provenir de una o de varias tablas.

TUIA - BASES DE DATOS I

VISTAS

- Las vistas suelen usarse para centrar, simplificar y personalizar la percepción de la base de datos para cada usuario.
- Las vistas pueden emplearse como mecanismos de seguridad, que permiten a los usuarios obtener acceso a los datos por medio de la vista, pero no les conceden el permiso de obtener acceso directo a las tablas base subyacentes de la vista.



TUIA - BASES DE DATOS I

VISTAS

- Las vistas pueden utilizarse para proporcionar una interfaz compatible con versiones anteriores con el fin de emular una tabla que existía, pero cuyo esquema ha cambiado.

TUIA - BASES DE DATOS I

VISTAS

```
USE [Northwind]
```

```
GO
```

```
CREATE VIEW [dbo].[ProductosListaAlfabetica] AS
```

```
SELECT Products.*, Categories.CategoryName
```

```
FROM Categories
```

```
INNER JOIN Products ON Categories.CategoryID = Products.CategoryID
```

```
WHERE (((Products.Discontinued)=0))
```

```
GO
```

TUIA - BASES DE DATOS I

VISTAS

```
USE [Northwind]
```

```
GO
```

```
ALTER VIEW [dbo].[ProductosListaAlfabetica] AS
```

```
SELECT Products.*, Categories.CategoryName
```

```
FROM Categories
```

```
INNER JOIN Products ON Categories.CategoryID = Products.CategoryID
```

```
WHERE (((Products.Discontinued)=1))
```

```
GO
```

TUIA - BASES DE DATOS I

FUNCIONES

¿Qué son las Funciones en MS-SQL?

Las funciones en MS-SQL son rutinas predefinidas o personalizadas que aceptan uno o más parámetros como entrada, realizan operaciones específicas y devuelven un valor como resultado. Pueden utilizarse en diversas partes de una consulta, como en la lista de selección (**SELECT**), la cláusula **WHERE** o en otras funciones.

TUIA - BASES DE DATOS I

FUNCIONES

Tipos de Funciones en MS-SQL:

Funciones Incorporadas (Built-in):

MS-SQL proporciona un conjunto de funciones integradas que realizan operaciones comunes. Ejemplos incluyen SUM(), AVG(), LEN(), etc.

TUIA - BASES DE DATOS I

FUNCIONES

Tipos de Funciones en MS-SQL:

Funciones Escalares (Scalar Functions):

Devuelven un solo valor basado en la entrada proporcionada. Por ejemplo, una función que concatena dos cadenas.



TUIA - BASES DE DATOS I

FUNCIONES

Tipos de Funciones en MS-SQL:

Funciones de Tabla (Table Functions):

Devuelven un conjunto de filas como resultado.

TUIA - BASES DE DATOS I

FUNCIONES

Tipos de Funciones en MS-SQL:

Funciones de Agregado (Aggregate Functions):

Realizan operaciones en un conjunto de valores y devuelven un solo valor resumido. Ejemplos son SUM(), COUNT(), AVG().

TUIA - BASES DE DATOS I

FUNCIONES

Tipos de Funciones en MS-SQL:

Funciones de Ventana (Window Functions):

Realizan cálculos en un conjunto de filas relacionadas con la fila actual.
Ejemplos son ROW_NUMBER(), RANK(), LEAD()..

TUIA - BASES DE DATOS I

FUNCIONES

-- Función Escalar

```
SELECT dbo.NombreFuncionEscalar(Columna) AS Resultado  
FROM Tabla;
```

-- Función de Tabla

```
SELECT *  
FROM dbo.NombreFuncionTabla(Columna);
```

-- Función de Agregado

```
SELECT AVG(Columna) AS Promedio  
FROM Tabla;
```

-- Función de Ventana

```
SELECT Columna, ROW_NUMBER() OVER (ORDER BY OrdenColumna) AS NumeroFila  
FROM Tabla;
```

TUIA - BASES DE DATOS I

FUNCIONES

Funciones Creadas por el Usuario en MS-SQL

Las funciones creadas por el usuario en MS-SQL son rutinas que los desarrolladores pueden definir para realizar operaciones específicas sobre los datos almacenados en la base de datos. Estas funciones personalizadas pueden simplificar consultas complejas, mejorar la modularidad del código y facilitar la reutilización de lógica de negocio.

TUIA - BASES DE DATOS I

FUNCIONES

Funciones Creadas por el Usuario en MS-SQL

Definición de la Función:

Se comienza definiendo la función con la palabra clave CREATE FUNCTION.

Se especifica el nombre de la función, los parámetros de entrada y el tipo de dato que devolverá.

TUIA - BASES DE DATOS I

FUNCIONES

Funciones Creadas por el Usuario en MS-SQL

```
CREATE FUNCTION dbo.NombreFuncion
```

```
(
```

```
    @Parametro1 TipoDato1,
```

```
    @Parametro2 TipoDato2
```

```
)
```

```
RETURNS TipoDatoResultado
```

```
AS
```

```
BEGIN
```

```
    -- Lógica de la función
```

```
    RETURN Resultado;
```

```
END;
```

TUIA - BASES DE DATOS I

FUNCIONES

Funciones Creadas por el Usuario en MS-SQL

Cuerpo de la Función:

En el bloque BEGIN y END, se incluye la lógica de la función. Puede implicar cálculos, manipulación de datos o cualquier otra operación requerida.

TUIA - BASES DE DATOS I

FUNCIONES

Funciones Creadas por el Usuario en MS-SQL

```
CREATE FUNCTION dbo.ConcatenarCadenas
```

```
(
```

```
    @Cadena1 NVARCHAR(MAX),
```

```
    @Cadena2 NVARCHAR(MAX)
```

```
)
```

```
RETURNS NVARCHAR(MAX)
```

```
AS
```

```
BEGIN
```

```
    RETURN @Cadena1 + @Cadena2;
```

```
END;
```

TUIA - BASES DE DATOS I

FUNCIONES

Uso de Funciones en Consultas

En la Cláusula SELECT: Las funciones pueden ser utilizadas en la lista de selección para modificar o calcular valores.

```
SELECT dbo.ConcatenarCadenas(Nombre, Apellido) AS NombreCompleto  
FROM Tabla;
```

TUIA - BASES DE DATOS I

FUNCIONES

Uso de Funciones en Consultas

En la Cláusula WHERE: Pueden ser parte de condiciones en la cláusula WHERE.

```
SELECT *  
FROM Tabla  
WHERE dbo.EdadMayorA18(Edad) = 1;
```


TUIA - BASES DE DATOS I

FUNCIONES

Ventajas de las Funciones Creadas por el Usuario:

- Reutilización de Código: Permite encapsular lógica compleja y reutilizarla en múltiples consultas.
- Modularidad: Mejora la estructura del código al dividirlo en funciones más pequeñas y manejables.
- Facilita el Mantenimiento: Cambios en la lógica solo necesitan realizarse en un lugar (en la función) en lugar de en múltiples consultas.

TUIA - BASES DE DATOS I

OVER

- La cláusula OVER define una ventana o un conjunto de filas definido por el usuario en un conjunto de resultados de la consulta.
- Determina las particiones y el orden de un conjunto de filas antes de que se aplique la función de ventana asociada.

TUIA - BASES DE DATOS I

OVER

- Una función de ventana calcula entonces un valor para cada fila de la ventana.
- Se puede utilizar la cláusula OVER con funciones para calcular valores agregados tales como medias móviles, agregados acumulados, totales acumulados o N elementos superiores por resultados del grupo.

TUIA - BASES DE DATOS I

OVER

Las funciones de ventana (Windows function) podrían tener los siguientes argumentos en su cláusula OVER:

- PARTITION BY, que divide el conjunto de resultados de la consulta en particiones.
- ORDER BY, que define el orden lógico de las filas dentro de cada partición del conjunto de resultados.
- ROWS/RANGE, que limita aún más las filas de la partición especificando los puntos inicial y final. Requiere el argumento ORDER BY y el valor predeterminado es desde el inicio de la partición al elemento actual si se especifica el argumento ORDER BY.

TUIA - BASES DE DATOS I

OVER

Las funciones de agregado actúan sobre toda la tabla y devuelven valores utilizando la cláusula GROUP BY.

Sin embargo, a veces es necesario utilizarlas sobre conjuntos más pequeños o ventanas.

TUIA - BASES DE DATOS I OVER

```
USE [SalesLT]
```

```
CREATE TABLE [Orders]
```

```
(  
   orderid INT,  
    orderdate DATE,  
    customerName VARCHAR(100),  
    City VARCHAR(50),  
    amount MONEY  
)
```

TUIA - BASES DE DATOS I OVER

```
INSERT INTO [Orders]
```

```
SELECT 1, '01/01/2021', 'Mohan Gupta', 'Alwar', 10000
```

```
UNION ALL
```

```
SELECT 2, '02/04/2021', 'Lucky Ali', 'Kota', 20000
```

```
UNION ALL
```

```
SELECT 3, '03/02/2021', 'Raj Kuman', 'Jaipur', 5000
```

```
UNION ALL
```

```
SELECT 4, '04/02/2021', 'Jyoti Kumari', 'Jaipur', 15000
```

```
UNION ALL
```

```
SELECT 5, '05/03/2021', 'Rahul Gupta', 'Jaipur', 7000
```

```
UNION ALL
```

```
SELECT 6, '06/04/2021', 'Mohan Kuman', 'Alwar', 25000
```

```
UNION ALL
```

```
SELECT 7, '07/02/2021', 'Kashish Agarwal', 'Alwar', 15000
```

```
UNION ALL
```

```
SELECT 8, '08/03/2021', 'Nagar Singh', 'Kota', 2000
```

```
UNION ALL
```

```
SELECT 9, '09/04/2021', 'Anil KG', 'Alwar', 1000
```

```
Go
```

TUIA - BASES DE DATOS I

OVER

Si queremos conocer el monto total de las órdenes a cada ciudad podemos usar GROUP BY y SUM

```
SELECT City AS CustomerCity  
, SUM(amount) AS totalamount  
FROM [Orders]  
GROUP BY city  
ORDER BY city
```


TUIA - BASES DE DATOS I

OVER

No podemos utilizar columnas no agregadas (que no estén en el GROUP BY) o en funciones de agregado

```
SELECT City AS CustomerCity,  
customerName,  
SUM(amount) AS totalamount  
FROM [Orders]  
GROUP BY city  
ORDER BY city
```

TUIA - BASES DE DATOS I

OVER – PARTITION BY

La cláusula PARTITION BY crea una ventana , realiza la agregación y la muestra.
También se pueden usar columnas no agregadas en el SELECT

```
SELECT City AS CustomerCity, CustomerName, amount,  
SUM(amount) OVER(PARTITION BY city) TotalOrderAmount  
FROM [Orders]
```

	CustomerCity	CustomerName	amount	TotalOrderAmount
1	Alwar	Mohan Gupta	10000,00	51000,00
2	Alwar	Mohan Kumar	25000,00	51000,00
3	Alwar	Kashish Agarwal	15000,00	51000,00
4	Alwar	Anil KG	1000,00	51000,00
5	Jaipur	Raj Kumar	5000,00	27000,00
6	Jaipur	Jyoti Kumari	15000,00	27000,00
7	Jaipur	Rahul Gupta	7000,00	27000,00
8	Kota	Lucky Ali	20000,00	22000,00
9	Kota	Nagar Singh	2000,00	22000,00

TUIA - BASES DE DATOS I

OVER – PARTITION BY

Podemos utilizar otras funciones de agregado:

```
SELECT City AS CustomerCity, CustomerName, amount,  
SUM(amount) OVER(PARTITION BY city) TotalOrderAmount,  
Avg(amount) OVER(PARTITION BY city) AvgOrderAmount,  
Min(amount) OVER(PARTITION BY city) MinOrderAmount,  
MAX(amount) OVER(PARTITION BY city) MaxOrderAmount  
FROM [Orders]
```

TUIA - BASES DE DATOS I

OVER – PARTITION BY – ROW_NUMBER()

Podemos utilizar la función ROW_Number() para opciones de paginado:

```
SELECT City AS CustomerCity, CustomerName, amount,  
ROW_NUMBER() OVER(PARTITION BY city ORDER BY amount DESC) AS [Row Number]  
FROM [Orders]
```

TUIA - BASES DE DATOS I

OVER – PARTITION BY – ROW_NUMBER()

Podemos utilizar CURRENT ROW y FOLLOWING para sumas acumulativas parciales:

```
SELECT City AS CustomerCity, CustomerName, amount,  
ROW_NUMBER() OVER(PARTITION BY city ORDER BY amount DESC) AS [Row Number],  
SUM(amount) OVER(PARTITION BY city ORDER BY amount DESC ROWS BETWEEN  
CURRENT ROW AND 1 FOLLOWING) AS CumulativeSUM  
FROM [Orders]
```

TUIA - BASES DE DATOS I

OVER – PARTITION BY – ROW_NUMBER()

Podemos utilizar CURRENT ROW y **PRECEDING** para sumas acumulativas :

```
SELECT City AS CustomerCity, CustomerName, amount,  
ROW_NUMBER() OVER(PARTITION BY city ORDER BY amount DESC) AS [Row Number],  
SUM(amount) OVER(PARTITION BY city ORDER BY amount DESC  
ROWS UNBOUNDED PRECEDING) AS CumulativeSUM  
FROM [Orders]
```

TUIA - BASES DE DATOS I

OVER – PARTITION BY – ROW_NUMBER()

Comparativa entre GROUP BY vs PARTITION BY:

GROUP BY	PARTITION BY
Devuelve una fila por grupo despues de agrupar los valores	Devuelve todas las filas del SELECT con los valores agregados.
No podemos usar columnas no agregadas en el SELECT.	Podemos usar columnas no agregadas en el SELECT.
Se filtra con HAVING	La clausula PARTITION puede tener predicados adicionales en el WHERE
GROUP BY se usa para agregados regulares	PARTITION BY se usa en agregados de funciones de ventana
No podemos utilizarlo para calcular numeros de fila o ranks.	Podemos utilizarlo para calcular numeros de fila o ranks.